先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 05 – Discriminative Classifiers II: Perceptrons and Decision Trees

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods, one learning question

Time	What we will explain and calculate
45 min	A perceptron score, mistake update, four-point training and separability.
45 min	Tree paths, weighted impurity and the best fruit-size threshold.
45 min	Pruning, leaf probabilities and a fair model comparison.

Learning goal

Given a wrong prediction, explain what the learner can change and what its model cannot express.

Connection

Lecture 04 defined a discriminant score. Today we learn its weights and compare it with conditional rules.

Two models describe different boundaries

	Perceptron	Decision tree
Prediction	Sign of $w^\top x + b$	Follow feature tests to a leaf
Geometry	One hyperplane	Several axis-aligned regions
Learning	Correct a misclassified sample	Choose a split with lower impurity
Capacity	Additive score in given features	Different conditions in different regions

Message features

An additive spam score suits several weak clues. A rule such as “urgent AND unfamiliar sender” expresses a conditional combination.

A score, a label and a probability answer different questions

A concrete observation

$x = (2, 1)$, $w = (0.5, -1)$ and $b = 0.2$ give $a = 0.5(2) - 1 + 0.2 = 0.2$. Predict the positive class.

Known negative label

If the true label is -1, the signed score is $m = -0.2$, so this is a training mistake.

Three meanings

The score describes the model's side and strength. The label identifies the observed class. A probability requires a separate probabilistic model or calibration.

Interpretation

A score 0.2 does not mean 20% positive probability. Multiplying every parameter by 10 preserves the predicted class but makes the score 2.

A score gives a side and a boundary

$$a(x) = w_1 x_1 + w_2 x_2 + b = w^\top x + b, \qquad \hat{y} = \begin{cases} +1, & a(x) > 0 \\ -1, & a(x) \le 0. \end{cases}$$

Geometry

$a(x) = 0$ is the boundary. The vector w is perpendicular to it. The bias b shifts it.

Calculation

$w = (-2, 0)$, $b = 1$: $a(0, 1) = 1$, $a(1, 0) = -1$, and the boundary is $x_1 = 0.5$.

One compact notation

$\tilde{x} = (x_1, x_2, 1)^\top$, $\tilde{w} = (w_1, w_2, b)^\top$, so $a(x) = \tilde{w}^\top \tilde{x}$.

Tie rule

At score zero, prediction uses a fixed rule. Training treats zero signed score as requiring an update.

The normal direction and the offset move the boundary

$$a(x) = 2x_1 + x_2 - 4, \qquad w = (2, 1),\ b = -4.$$

Two boundary points

$(0, 4)$ and $(2, 0)$ both have score 0. Their difference $(2, -4)$ is perpendicular to w because $(2, 1)^\top (2, -4) = 0$.

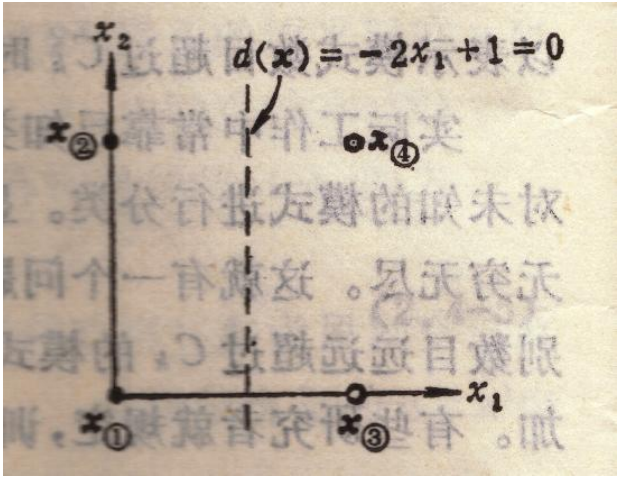
Offset change

Changing b from -4 to -6 gives $2x_1 + x_2 = 6$. The direction stays the same, but the line shifts.

Scale change

$4x_1 + 2x_2 - 8 = 0$ defines the original line. Parameter magnitude alone does not identify a different classifier.

The normal direction and the offset move the boundary



Augmentation retains the bias in every calculation

Original form

$w = (1, -2)$, $b = 0.5$, $x = (3, 1)$ give $a = 3 - 2 + 0.5 = 1.5$.

$$\tilde{x} = (3, 1, 1)^\top, \quad \tilde{w} = (1, -2, 0.5)^\top, \quad \tilde{w}^\top \tilde{x} = 1.5.$$

On a positive mistake

$\eta = 0.2$ would add $(0.6, 0.2, 0.2)$ to the augmented weights. The last 0.2 changes the bias.

Dimension check

Two measured inputs require three augmented coordinates. Appending 0 would remove the bias term. Appending a constant other than 1 requires a corresponding parameter convention.

The signed score identifies a mistake

$$m_i = y_i a(x_i), \qquad y_i \in \{-1, +1\}.$$

y_i	$a(x_i)$	m_i	Training interpretation
$+1$	2	2	Correct side
-1	-3	3	Correct side
-1	2	-2	Mistake
$+1$	0	0	On the boundary: update

Perceptron criterion

$J(w) = \sum_{i:m_i \le 0}(-m_i)$. A negative signed score contributes a positive penalty.

Interpretation

The score is not a calibrated probability. The criterion measures signed errors, not the count of errors.

Counting mistakes and measuring their depth differ

Two candidate models

ModelA has signed scores $(-0.2, -3, 2)$. ModelB has $(-1, -1, 1)$. Both misclassify two of the three examples.

$$J_A = 0.2 + 3 = 3.2, \qquad J_B = 1 + 1 = 2.$$

Criterion reading

The perceptron penalty distinguishes how far wrong examples fall on the wrong side. The 0–1 error count ignores this depth.

Boundary case

All-zero weights give signed score 0 everywhere. Our algorithm explicitly updates zero scores, rather than accepting that degenerate state as a useful separator.

A mistake gives an update direction

For one misclassified example

$\ell_i = -y_i \tilde{\boldsymbol{w}}^\top \tilde{\boldsymbol{x}}_i$ and $\nabla_{\tilde{\boldsymbol{w}}} \ell_i = -y_i \tilde{\boldsymbol{x}}_i$.

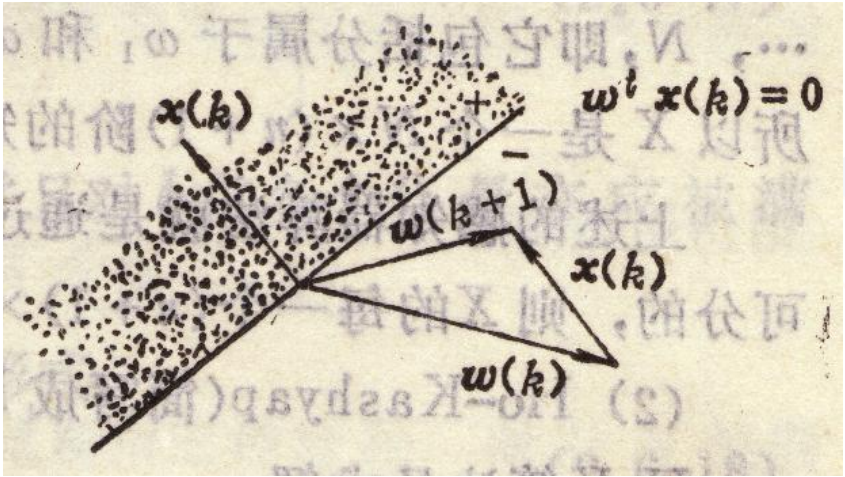
$$\tilde{\boldsymbol{w}}_{\text{new}} = \tilde{\boldsymbol{w}} - \eta \nabla \ell_i = \tilde{\boldsymbol{w}} + \eta y_i \tilde{\boldsymbol{x}}_i, \qquad \eta > 0.$$

Condition	Action
$y_i a(\boldsymbol{x}_i) > 0$	Keep the weights.
$y_i a(\boldsymbol{x}_i) \leq 0$	Add $\eta y_i \boldsymbol{x}_i$ to $\boldsymbol{w}$ and add ηy_i to b.

Negative mistake

$\boldsymbol{x} = (1, 0), y = -1, \eta = 1$ gives $\Delta \boldsymbol{w} = (-1, 0)$ and $\Delta b = -1$.

A mistake gives an update direction



Batch and sequential updates can follow different paths

Two positive examples

$\tilde{x}_A = (1, 0, 1)$ and $\tilde{x}_B = (1, 1, 1)$, with $\tilde{w} = 0$ and $\eta = 1$. Both initially have signed score 0.

Batch step

Using the initial active set gives $\Delta\tilde{w} = \tilde{x}_A + \tilde{x}_B = (2, 1, 2)$.

Sequential step

Update onA to $(1, 0, 1)$. B's new score is 2, so skip its update. The final weights are $(1, 0, 1)$.

Meaning

The sequential method recomputes each example's score after earlier changes. A batch step uses errors evaluated at the same parameter state. Both are separators here.

Why the corrected sample improves

$$
\begin{aligned}
y_i \tilde{w}_{\text{new}}^{\top} \tilde{x}_i &= y_i (\tilde{w} + \eta y_i \tilde{x}_i)^{\top} \tilde{x}_i \\
&= y_i \tilde{w}^{\top} \tilde{x}_i + \eta \|\tilde{x}_i\|^2 .
\end{aligned}
$$

Direction check

Its signed score increases because $y_i^2 = 1$ and $\eta \|\tilde{x}_i\|^2 > 0$. One update can still leave it misclassified.

Calculation

$\tilde{x} = (1, 0, 1)$ and $\eta = 1$ increase the signed score by 2. A starting score of -5 becomes -3.

Other examples

For another sample j, the change is $\eta y_i y_j \tilde{x}_i^{\top} \tilde{x}_j$. It may help or hurt that sample.

Correcting one example can disturb another

Before the update

$\tilde{w} = (1, 0, 1)$. PositiveA has $\tilde{x}_A = (0, 0, 1)$ and score 1. NegativeB has $\tilde{x}_B = (1, 2, 1)$ and score 2, so B is wrong.

$$\tilde{w}' = \tilde{w} - \tilde{x}_B = (0, -2, 0).$$

After the update

B's score becomes -4, giving signed score 4. A's score falls to 0. The correction helpsB but removesA's positive margin.

Why repeat epochs

Parameters are shared across all observations. A later update can invalidate an earlier prediction, so stopping checks the entire training set.

Four points make the training visible

Sample	(x_1, x_2)	$\tilde{x}$	y
A	$(0, 0)$	$(0, 0, 1)$	$+1$
B	$(0, 1)$	$(0, 1, 1)$	$+1$
C	$(1, 0)$	$(1, 0, 1)$	-1
D	$(1, 1)$	$(1, 1, 1)$	-1

Training settings

$\tilde{w}^{(0)} = (0, 0, 0)$, $\eta = 1$, visit order A, B, C, D. Update whenever $y\tilde{w}^{\top}\tilde{x} \leq 0$.

Before calculating

The label depends only on x_1. We expect a vertical separating boundary, but the algorithm must discover it.

A different visit order finds a different valid separator

Epoch	OrderC, A, D, B: end weights	Updates
1	$(-1, 1, 1)$	3
2	$(-3, 1, 1)$	4
3	$(-3, 1, 1)$	0

Same four observations

Keep labels and learning rate from the original example. Only the visit order changes.

$$a(x) = -3x_1 + x_2 + 1.$$

Verify

A, B, C, D now have scores 1, 2, -2, -1. Every signed score is positive. The boundary is tilted rather than vertical, though both separators fit these four points.

Epoch 1 changes the bias and the first weight

Visit	Weights before	Signed score	Update	Weights after
A	$(0, 0, 0)$	0	$+(0, 0, 1)$	$(0, 0, 1)$
B	$(0, 0, 1)$	1	none	$(0, 0, 1)$
C	$(0, 0, 1)$	-1	$-(1, 0, 1)$	$(-1, 0, 0)$
D	$(-1, 0, 0)$	1	none	$(-1, 0, 0)$

What changed

A first creates a positive bias. C then lowers its own score by changing w_1 and b.

End of epoch 1

A and B now lie on score zero. This epoch contained two updates, so training continues.

Later epochs produce a verified solution

Epoch	Updates during this pass	Final weights
2	A: $(-1,0,0) \to (-1,0,1)$; C: $(-1,0,1) \to (-2,0,0)$	$(-2,0,0)$
3	A: $(-2,0,0) \to (-2,0,1)$; B, C, D already correct	$(-2,0,1)$
4	No update on A, B, C, D	$(-2,0,1)$

$$a(A) = a(B) = 1, \qquad a(C) = a(D) = -1.$$

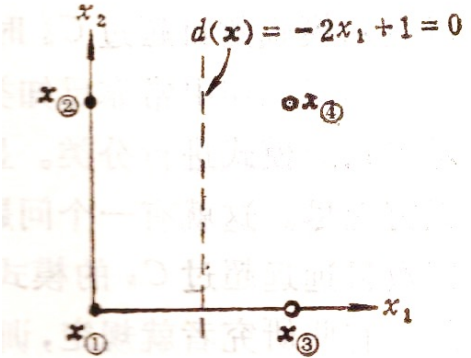
Stopping condition

A complete pass has zero updates. The solution is $a(x) = -2x_1 + 1$, with boundary $x_1 = 0.5$.

Exercise

Explain why epoch 3 finds the solution but epoch 4 confirms the algorithm's stopping condition.

The learned score defines a vertical boundary



Read the axes

x_1 is horizontal and x_2 is vertical. The dashed line is $-2x_1 + 1 = 0$.

A new point

$x = (0.25, 0.8)$ gives $a = 0.5$, so predict $+1$. At $x_1 = 0.75$, the score is -0.5.

Feature meaning

Changing x_2 has no effect for this learned model because $w_2 = 0$.

A short implementation matches the hand calculation

```
w = np.zeros(X.shape[1])
b = 0.0
for epoch in range(max_epochs):
    mistakes = 0
    for x, y in zip(X, labels):
        if y * (w @ x + b) <= 0:
            w += eta * y * x
            b += eta * y
            mistakes += 1
    if mistakes == 0:
        break
```

Checks

Labels are $-1, +1$. Both w and b update. A whole zero-update epoch triggers stopping. Keep a maximum epoch limit for nonseparable data.

Two zero-error classifiers can disagree on a new point

Two learned scores

$a_1 = -2x_1 + 1$ and $a_2 = -3x_1 + x_2 + 1$ both classify all four training observations correctly.

New observation

$x = (0.6, 1)$ gives $a_1 = -0.2$ and $a_2 = 0.2$. The first model predicts negative and the second positive.

Evidence needed

The training set contains only $x_1 = 0$ or 1, so it cannot settle the label at 0.6 by itself. More representative data or a justified modeling preference is required.

Conclusion

Training accuracy 100% identifies agreement on observed points. It does not prove agreement or correctness throughout the input space.

The path depends on order and feature scale

- Earlier updates change the scores seen by later examples. A different visit order may reach a different separating hyperplane.
- With zero initialization and fixed positive η, multiplying η scales all visited weights. Signs follow the same path under exact arithmetic.
- Changing a feature from metres to millimetres changes its update size and the model's geometry. Training-only scaling can help.
- Several separators can fit the same finite data. Zero training error does not identify the best separator for future data.

Unit example

An input 1000 times larger contributes 1000 times more to that coordinate's update. This is a representation effect, not new information.

Averaging visited weights can soften oscillation

Illustrative visits

With zero biases, three weight vectors are $(1, 0), (-1, 2), (1, 2)$. Their average is $(1/3, 4/3)$.

One prediction

At $x = (3, 1)$, visited scores are $3, -1, 5$, while the averaged score is $7/3$.

Averaged perceptron

Average parameter states over training visits. This differs from pocket learning, which retains the single visit with the smallest observed error.

Scope

Averaging can reduce sensitivity to the last few updates. It does not guarantee the best separator or calibrated probabilities. The sequence shown here illustrates the operation rather than a claimed experiment.

Convergence requires a separable training set

Assumption

There exists a unit vector u and $\gamma > 0$ such that $y_i u^\top \tilde{x}_i \geq \gamma$ for every training example.

Result

For a finite dataset, the perceptron makes finitely many mistake updates under this condition. It eventually reaches a zero-update epoch.

What it guarantees

A separator for the given training examples. The four-point example satisfies the assumption.

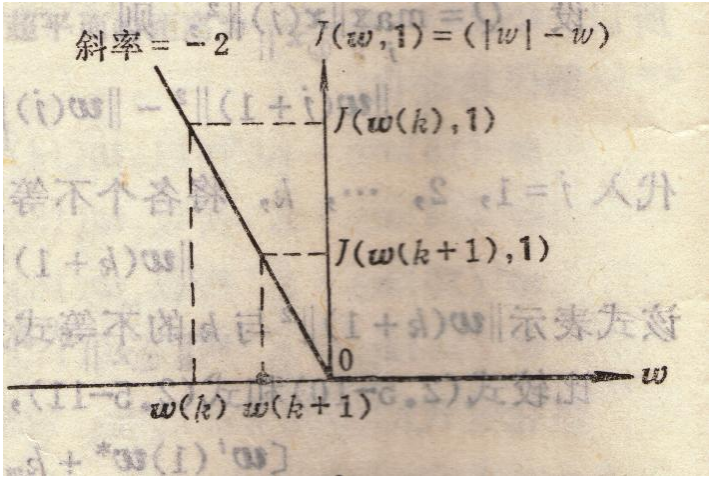
What still needs evidence

Generalization on new samples, probability calibration and the size of the learned margin do not follow from this theorem.

Diagnosis

Repeated updates can indicate inseparability, label errors, poor representation or an implementation mistake.

Convergence requires a separable training set



Slow convergence and impossible separation are different

Training situation	What the symptom can mean
Same input with opposite labels	Strict separation is impossible.
Separable points with a tiny margin	Updates may continue for many visits.
Bias or labels implemented incorrectly	The program may fail even on easy data.

Concrete contradiction

One input $x = (1, 1)$ cannot have both $a(x) > 0$ and $a(x) < 0$. More epochs cannot satisfy both labels.

Diagnosis order

Check the implementation on a known toy set, inspect conflicting records, then examine representation and training budget. Repeated errors alone do not identify their cause.

A larger margin permits fewer mistakes

Notation

Use $\eta = 1$, zero initialization, $\|\tilde{x}_i\| \le R$, and separator margin $y_i u^\top \tilde{x}_i \ge \gamma$, $\|u\| = 1$.

$$
\begin{aligned}
u^\top \tilde{w}_M &\ge M\gamma, \\
\|\tilde{w}_{t+1}\|^2 &= \|\tilde{w}_t\|^2 + 2y_i \tilde{w}_t^\top \tilde{x}_i + \|\tilde{x}_i\|^2 \le \|\tilde{w}_t\|^2 + R^2, \\
M\gamma \le \|\tilde{w}_M\| &\le R\sqrt{M} \quad \Rightarrow \quad M \le (R/\gamma)^2.
\end{aligned}
$$

Scale of the bound

$R = 2, \gamma = 0.5$ gives $M \le 16$. With $\gamma = 0.1$, the bound becomes 400. This is a bound on updates, not epochs.

The four-point example satisfies the mistake bound

Augmented data bound

The largest augmented squared length is 3, attained by $(1, 1, 1)$, so $R = \sqrt{3}$.

$$u = \frac{(-2, 0, 1)}{\sqrt{5}}, \qquad \gamma = \min_i y_i u^\top \tilde{x}_i = \frac{1}{\sqrt{5}}.$$

$$M \le \left(\frac{R}{\gamma}\right)^2 = \frac{3}{1/5} = 15.$$

Observed updates

The original order made 2, 2, 1, 0 updates across four epochs, totaling 5. Five is within the bound 15.

Counting

The bound applies to mistake updates. Four epochs visited 16 examples but did not perform 16 updates.

Functional and geometric margins use different normalizations

Raw-input distance

For $a = -2x_1 + 1$, positive $(0, 1)$ has signed score 1. Its Euclidean distance to the line $x_1 = 0.5$ is $1/\|w\| = 1/2$.

Augmented-space normalization

In the convergence proof, the parameter includes bias and has norm $\sqrt{5}$. The same signed score normalizes to $1/\sqrt{5}$.

Scale check

Multiplying w, b by 10 makes the score 10, but raw geometric distance remains $10/20 = 0.5$.

Meaning

State which vector norm and coordinate space define the margin. A raw score can grow without moving the decision boundary.

XOR reveals a limitation of one linear score

x_1	x_2	y
0	0	-1
0	1	$+1$
1	0	$+1$
1	1	-1

Required inequalities

$b < 0$, $w_2 + b > 0$, $w_1 + b > 0$, $w_1 + w_2 + b < 0$.

Contradiction

The middle two require $w_1 + w_2 + b > -b > 0$, which contradicts the last inequality.

A useful feature

$z = x_1 + x_2 - 2x_1x_2$ equals 1 for the positive points and 0 otherwise. A nonlinear representation can change separability.

An explicit interaction makes XOR linearly separable

x_1	x_2	$z = x_1 + x_2 - 2x_1x_2$	$a = 2z - 1$
0	0	0	-1
0	1	1	1
1	0	1	1
1	1	0	-1

New score

The rule is linear in z, but $a = 2x_1 + 2x_2 - 4x_1x_2 - 1$ is nonlinear in the original inputs.

What changed

The learner's linear form remains. The representation now exposes an interaction. A transformed-feature solution requires a defensible construction and validation on new data.

Nonseparable data needs a practical stopping rule

A training budget

Set a maximum number of epochs. Count training mistakes after each candidate update or epoch.

Pocket rule

Keep the weights with the smallest observed training error. Later worse weights do not overwrite them.

Example

Epoch errors 12, 8, 10, 7, 9 leave the pocket holding epoch 4, not the final epoch.

Evaluation

Use validation data to choose model settings and report test performance once. Pocket search does not prove optimality or repair label noise.

Alternatives

Better features, regularized linear models or a model with nonlinear boundaries can address the actual cause.

Multiclass perceptron corrects two class scores

$$a_c(x) = \tilde{w}_c^\top \tilde{x}, \qquad \hat{y} = \arg\max_c a_c(x).$$

On a mistake

$\tilde{w}_y \leftarrow \tilde{w}_y + \eta\tilde{x}$, $\tilde{w}_{\hat{y}} \leftarrow \tilde{w}_{\hat{y}} - \eta\tilde{x}$. Other rows stay unchanged.

Calculation

For scores $(1, 3, 0)$ and true class 1, class 2 wins. With $\eta = 1$ and $\|\tilde{x}\|^2 = 2$, class 1 rises to 3 and class 2 falls to 1 on that sample.

Connection

The true score must exceed competing scores. One-versus-rest instead trains separate binary tasks and needs a score-combination rule.

A multiclass mistake changes a score gap

Three weight rows

$\tilde{w}_1 = (1, 1, 0), \tilde{w}_2 = (0, 0, 0), \tilde{w}_3 = (-1, 0, 0)$. Input $\tilde{x} = (1, 0, 1)$ truly belongs to class 2.

Before

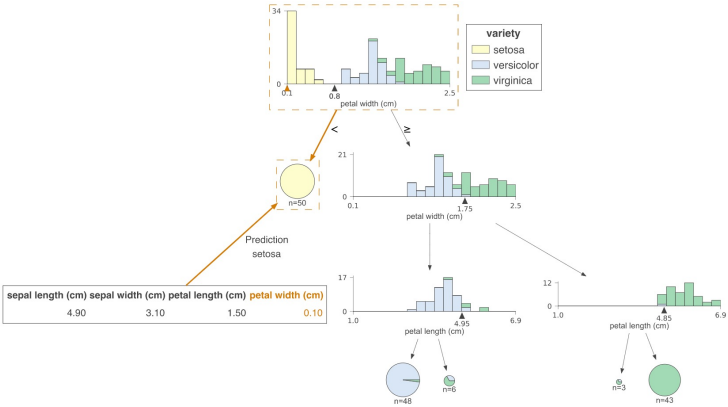
Scores $(1, 0, -1)$ choose class 1. With $\eta = 1$, subtract $\tilde{x}$ from row 1 and add it to row 2.

$$\tilde{w}_1' = (0, 1, -1), \quad \tilde{w}_2' = (1, 0, 1), \quad \tilde{w}_3' = (-1, 0, 0).$$

After

Scores become $(-1, 2, -1)$. The true-versus-wrong gap rises from -1 to 3, an increase of $2\|\tilde{x}\|^2 = 4$.

A tree prediction is a path of questions



Trace one sample

Petal width 0.10 cm is below 0.8 cm. The left leaf predicts setosa.

Follow a different path

Width 1.9 cm passes both width tests. The right branch then asks about petal length.

Conditional decisions

Bars show label mixtures. A later test applies only to samples that arrived at that node.

Paths create conditional regions

Path	Region and prediction
$x_1 \leq 2$	Predict class A.
$x_1 > 2$ and $x_2 \leq 5$	Predict class B.
$x_1 > 2$ and $x_2 > 5$	Predict class C.

Geometry

The root divides the full space. A later split divides only the region inherited from its ancestors. Leaves cover disjoint regions.

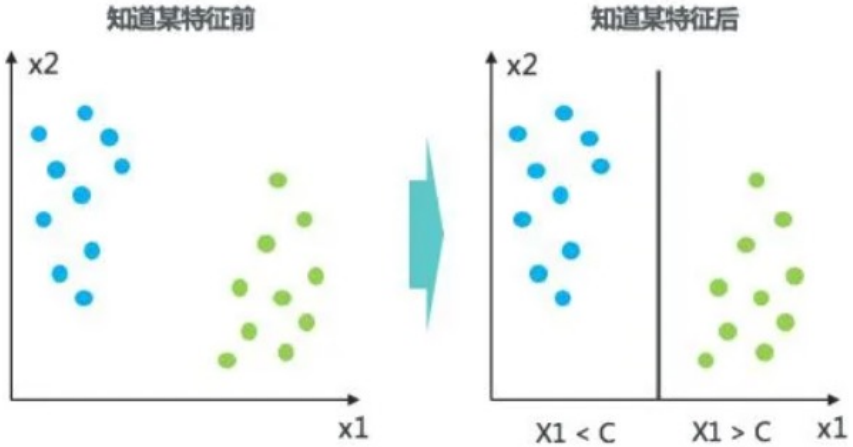
Calculation

For $(3, 4)$, the root answer is false and the second answer is true, giving class B. For $(1, 100)$, the root alone gives A.

Scope

A rule learned in a small region is not evidence of a global causal relationship.

Paths create conditional regions



A node receives a subset, not the entire dataset

Fruit width	Class	Child at 6.55
6.0	Lemon	Left
6.5	Lemon	Left
6.6	Orange	Right
6.9	Orange	Right
6.9	Lemon	Right
7.0	Orange	Right

Local dataset

The left child contains two lemons. The right child contains three oranges and one lemon. Only the right four observations participate in its next split search.

Denominators

The right node's class probabilities use denominator 4. Its contribution to whole-tree impurity additionally uses the fraction 4/6.

A leaf rule is a conjunction of all ancestor tests

Three illustrative paths

A:$x_1 \le 2$. B:$x_1 > 2$ and $x_2 \le 5$. C:$x_1 > 2$ and $x_2 > 5$.

$$\hat{y}(x) = \begin{cases} A, & x_1 \le 2 \\ B, & x_1 > 2,\ x_2 \le 5 \\ C, & x_1 > 2,\ x_2 > 5. \end{cases}$$

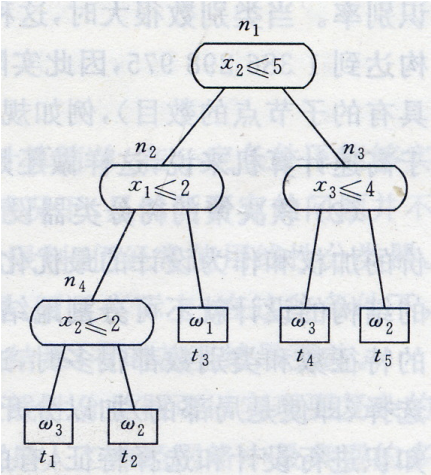
Trace three cases

$(1, 100)$ goes toA,$(3, 4)$ toB,$(3, 6)$ toC. The second question applies only after the first rejectsA.

Logical meaning

Each path is anAND of conditions. Combining the paths for one class is anOR. This turns a fitted tree into explicit rules with a stated scope.

A leaf rule is a conjunction of all ancestor tests



A leaf estimates a label distribution

$$\hat{p}(c \mid x \in R) = \frac{n_{R,c}}{n_R}, \qquad \hat{y} = \arg\max_c \hat{p}(c \mid R).$$

Classification leaf

With counts $(6, 3, 1)$, the leaf probabilities are $(0.6, 0.3, 0.1)$ and the predicted class is the first.

Regression leaf

For targets $(2, 3, 7)$, the squared-error prediction is their mean, 4. Its within-leaf sum of squares is $4 + 1 + 9 = 14$.

Sample size

A fraction of 1 from two examples carries much less evidence than a fraction of 0.9 from 100 examples. Leaf purity and reliable probability are different questions.

The same predicted class can have different support

Leaf counts(+, -)	Positive fraction	Predicted class
8, 2	0.80	Positive
6, 4	0.60	Positive
2, 0	1.00	Positive

Different evidence

The first two leaves agree on the majority label but differ in mixture. The third is pure yet has only two observations.

Decision question

If missing a positive is costly, a majority-label rule may be insufficient. A probability threshold and relevant costs must be defined separately.

Reliability

Training class fractions are empirical estimates. Neither purity nor a displayed decimal guarantees calibration on future observations.

A split must name a feature and a threshold

Numeric candidate

$x_j \leq t$ sends a sample left. Sort unique values and test midpoints between adjacent values.

Values with a tie

$6.0, 6.5, 6.6, 6.9, 6.9, 7.0$ give candidate thresholds $6.25, 6.55, 6.75, 6.95$. Equal values must stay together.

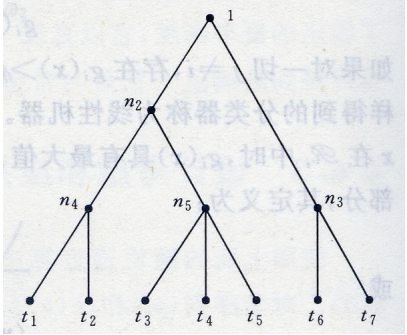
Training decision

For each candidate, count child labels and compute the sample-weighted child impurity. Choose the lowest value among admissible candidates.

Constraints

A candidate can be rejected if a child has too few examples. Changing the threshold within an empty gap does not change training membership.

Recursive growth repeats one local decision



1. Receive the current node's dataset D.
2. If pure or stopped, create a leaf.
3. Compare admissible feature/threshold splits using weighted child impurity.
4. Partition D and repeat in each child.

Read the source tree

Root 1, internal nodes $n_2, n_3, \ldots$, and leaves $t_1, \ldots, t_7$ distinguish decisions from terminal answers.

Greedy search

The best immediate split need not begin the globally best tree. Each descendant sees only its inherited samples.

Restrictions

Depth limits sequential questions. Minimum leaf size limits how little evidence can support a leaf.

Entropy measures a node's label uncertainty

$$H(D) = -\sum_{c=1}^{C} p_c \log_2 p_c, \qquad p_c = n_c/n, \quad 0 \log_2 0 := 0.$$

Binary counts	Class probabilities	Entropy in bits
$(10, 0)$	$(1, 0)$	0
$(5, 5)$	$(0.5, 0.5)$	1
$(7, 3)$	$(0.7, 0.3)$	0.8813

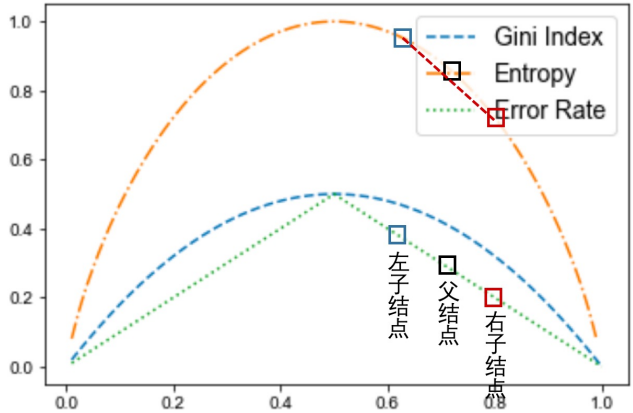
Reading

A pure node leaves no label uncertainty. A balanced binary node has maximum uncertainty. The base changes units, not the ordering of split gains.

Training use

A split should lower the average uncertainty among its children, accounting for how many samples each child receives.

Entropy measures a node's label uncertainty



Entropy 7-to 3 can be calculated term by term

Ten labels

Seven positives and three negatives give $p_+ = 0.7$ and $p_- = 0.3$.

$$\begin{aligned} H &= -0.7\log_2(0.7) - 0.3\log_2(0.3) \\ &\approx 0.7(0.514573) + 0.3(1.736966) \\ &= 0.360201 + 0.521090 = 0.881291 \text{ bits.} \end{aligned}$$

Interpretation

The rarer class has a larger surprise per occurrence, but its contribution also includes its probability. A pure node has no uncertainty.

Numerical handling

Use $0 \log 0 = 0$ by limit. Round for display only after calculating the weighted sum.

Conditional entropy must include child membership

Parent counts

Six positives and four negatives give $H(D) \approx 0.970951$ bits. A split creates child counts $(2, 0)$ and $(4, 4)$.

$$H(Y \mid \text{child}) = \frac{2}{10}(0) + \frac{8}{10}(1) = 0.8.$$

$$\text{Gain} = 0.970951 - 0.8 = 0.170951 \text{ bits.}$$

Meaning

Knowing the child removes some uncertainty, but 80% of cases still enter a balanced child. An unweighted mean of child entropies would incorrectly give 0.5.

Connection

Entropy gain and Gini decrease share the same sample-weighted structure. They differ in the impurity function.

Gini gives another measure of mixture

$$G(D) = 1 - \sum_c p_c^2. \qquad \text{Binary: } G = 2p(1-p).$$

Counts	Gini	Majority-class error
$(10, 0)$	0	0
$(5, 5)$	0.5	0.5
$(7, 3)$	0.42	0.3

Interpretation

Two independently drawn labels from the empirical node distribution disagree with probability G.

Choice

Entropy and Gini usually prefer purer children but may choose different splits. Gini uses no logarithm. Training software must use the same criterion for parent and children.

Multiclass impurity uses all class proportions

Eight observations

Counts $(4, 2, 2)$ give probabilities $(0.5, 0.25, 0.25)$. The majority-class prediction still chooses the first class.

$$G = 1 - 0.5^2 - 0.25^2 - 0.25^2 = 0.625.$$
$$H = -0.5(-1) - 0.25(-2) - 0.25(-2) = 1.5 \text{ bits}.$$

Error rate

Always predicting the first class makes 4/8 errors, so $E = 0.5$. The three quantities are different.

Bounds

Binary Gini's maximum 0.5 and binary entropy's maximum 1 do not apply to every multiclass node.

The best split lowers weighted child impurity

$$I_{\text{after}} = \frac{n_L}{n} I(D_L) + \frac{n_R}{n} I(D_R), \qquad \Delta I = I(D) - I_{\text{after}}.$$

Unequal children

Parent counts $(6, 4)$ have Gini 0.48. Split into a pure child $(2, 0)$ and child $(4, 4)$:

$$I_{\text{after}} = \frac{2}{10} \cdot 0 + \frac{8}{10} \cdot 0.5 = 0.4, \qquad \Delta I = 0.08.$$

Common error

A simple average gives 0.25 and exaggerates the split's benefit. A tiny pure leaf cannot represent the other 80% of examples.

General case

Use the same sample weighting for more than two children. Class weights replace raw counts with consistent weighted counts.

Entropy and Gini both compare candidate partitions

Candidate	Children(+, -)	Weighted Gini	Weighted entropy
A	(4, 0);(2, 4)	0.2667	0.5510
B	(3, 1);(3, 3)	0.4500	0.9245

Shared parent

Counts $(6, 4)$ have Gini 0.48 and entropy 0.9710. A gives the larger decrease under both measures in this example.

One calculation

For A, $G_{\text{after}} = \frac{4}{10}(0) + \frac{6}{10}[2(1/3)(2/3)] = 4/15$.

Scope

The two criteria often agree but need not rank every possible pair identically. Calculate each from its own definition and weights.

A tiny pure child is only one part of the gain

Parent $(7, 3)$

Gini is 0.42. Isolating one positive gives children $(1, 0)$ and $(6, 3)$.

$$G_{\text{after}} = \tfrac{1}{10}(0) + \tfrac{9}{10}\,[2(2/3)(1/3)] = 0.4, \quad \Delta G = 0.02.$$

Isolating one negative

Children $(0, 1)$ and $(7, 2)$ instead give $G_{\text{after}} = \frac{9}{10}\frac{28}{81} \approx 0.3111$. The gain is about 0.1089.

Meaning

A small pure child can have little or considerable gain depending on how it changes the large remainder. Purity alone cannot rank splits.

The Chinese fruit example gives real candidates

Width	Height	Class
7.0	9.5	Orange
6.9	6.0	Orange
6.6	6.6	Orange
6.5	6.6	Lemon
6.0	6.6	Lemon
6.9	9.6	Lemon

Parent

Three oranges and three lemons give $G(D) = 0.5$.

Width thresholds

$6.25, 6.55, 6.75, 6.95$.

Height thresholds

$6.30, 8.05, 9.55$.

Before calculating

The width 6.9 occurs in both classes. A width split cannot separate those two examples.

Question

Which feature and threshold produce the smallest weighted child Gini? Compare all seven admissible candidates.

Sorted counts evaluate all width thresholds

After width	Left(O, L)	Right(O, L)	Threshold
6.0	0, 1	3, 2	6.25
6.5	0, 2	3, 1	6.55
6.6	1, 2	2, 1	6.75
6.9	2, 3	1, 0	6.95

Cumulative scan

Move a whole equal-value group from right to left. Right counts equal parent counts minus left counts.

Efficiency

After sorting, the scan reuses counts instead of recounting every candidate from scratch. The equal-width 6.9 orange and lemon move together.

Check

Every row must sum to the original $(3, 3)$ class totals.

Width 6.55 wins the threshold comparison

Feature	Threshold and weighted child Gini			
Width	6.25 : 0.400	6.55 : 0.250	6.75 : 0.444	6.95 : 0.400
Height	6.30 : 0.400	8.05 : 0.500	9.55 : 0.400	—

Winning split

Width ≤ 6.55: (0 orange, 2 lemon). Width > 6.55: (3 orange, 1 lemon).

$$G_{\text{after}} = \frac{2}{6}(0) + \frac{4}{6}\left[1 - (3/4)^2 - (1/4)^2\right] = 0.25.$$

Gain

$0.5 - 0.25 = 0.25$. The right child is still mixed, so recursive growth may ask another question there.

The right fruit node can split again

Right-node observations

Widths above 6.55 leave heights 6.0, 6.6, 9.5 for three oranges and 9.6 for one lemon. The local class fractions are 3/4 and 1/4.

$$G = 2(3/4)(1/4) = 0.375.$$

Result

The right child is still mixed.

Next test

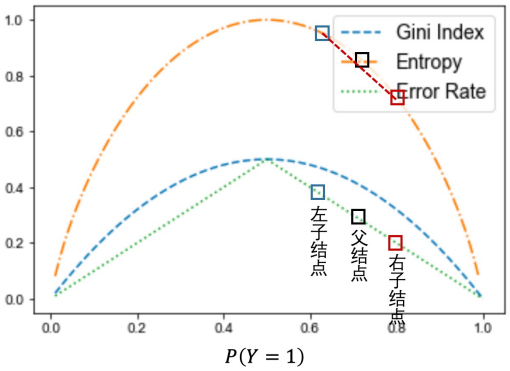
$\text{height} \leq 9.55$ sends all three oranges left and the one lemon right. Both local children are pure.

$$\Delta G_{\text{local}} = 0.375, \qquad \Delta G_{\text{whole}} = \frac{4}{6}(0.375) = 0.25.$$

Training versus future

Together with the first split, this tree fits all six fruits. The singleton lemon branch still needs evidence and complexity control for future fruit.

Error rate can miss a useful split



Same majority in both children

Parent $(7,3)$ splits into $(3,2)$ and $(4,1)$. Parent error is 0.30 and Gini is 0.42.

$$E_{\text{after}} = \frac{0.4 + 0.2}{2} = 0.30.$$

$$G_{\text{after}} = \frac{0.48 + 0.32}{2} = 0.40.$$

Reading the curve

Error is linear on one side of 0.5. Gini and entropy detect a change in class mixture even when both leaves predict the same majority class.

A split can change uncertainty without changing the label

Original source counts

Parent $(7, 3)$ splits into equal-size children $(3, 2)$ and $(4, 1)$. All three nodes predict positive.

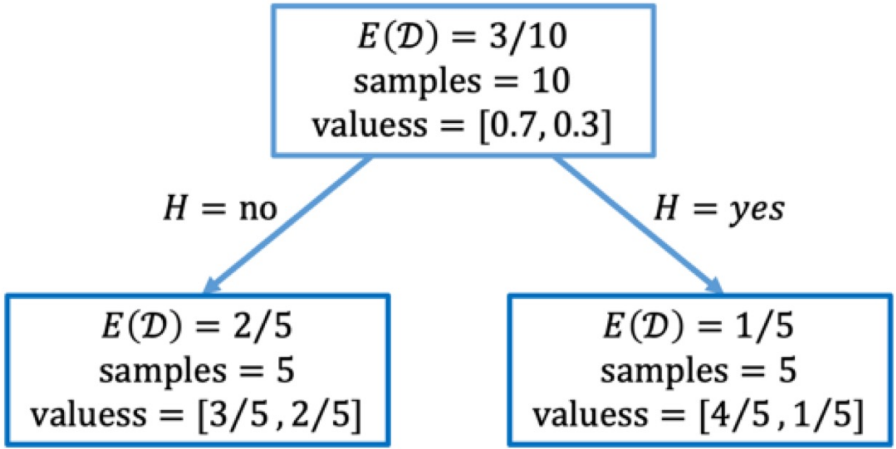
$$E_{\text{after}} = \tfrac{1}{2}(0.4) + \tfrac{1}{2}(0.2) = 0.3 = E_{\text{before}}.$$

$$H_{\text{after}} \approx \tfrac{1}{2}(0.9710) + \tfrac{1}{2}(0.7219) = 0.8464 < 0.8813.$$

What the question adds

Knowing the child changes the positive fraction from 0.7 to 0.6 or 0.8. This matters for subsequent splits or probability estimates even though majority-label decisions remain unchanged.

A split can change uncertainty without changing the label



Categorical splits ask membership questions

Nominal categories

For ward A, B, C, a binary split may ask whether ward belongs to $\{A, C\}$. Numeric codes 1, 2, 3 would impose an arbitrary order.

ID3 and C4.5

ID3 compares information gain for categorical branches. C4.5 uses gain ratio to reduce the preference for many-valued features.

$$\mathrm{GainRatio}(X) = \frac{H(Y) - H(Y \mid X)}{-\sum_k (n_k/n) \log_2 (n_k/n)}.$$

Identifiers

A unique ID can make every training child pure while teaching nothing about a new ID. A small denominator also needs careful candidate handling.

CART

Uses binary partitions. The supported handling of categorical and missing values depends on the implementation.

A missing split feature needs a routing convention

Fitted question

Suppose a tree first tests lab ≤ 5. A new row has a missing lab, so the numerical comparison alone cannot determine its branch.

Possible conventions

A model may use a learned missing-value branch, a surrogate feature test or training-fitted imputation before prediction. Supported behavior depends on the implementation.

Explicit baseline

If the training median is 4, median imputation sends this row left. A missing indicator can distinguish it from a real measurement 4.

Validity

Do not choose the branch after seeing the row's true label. Save the chosen rule and apply it consistently to future records.

Regression splits reduce squared error

Leaf prediction

$\bar{y}_R$ minimizes $\sum_{i \in R}(y_i - c)^2$. A split compares the sum of both children's squared errors.

Ordered observations

At $x = (1, 2, 3, 4)$, targets are $(1, 2, 8, 9)$. The parent mean is 5 and parent SSE is 50.

$$x \le 2.5: \quad \bar{y}_L = 1.5,\ \bar{y}_R = 8.5, \qquad \mathrm{SSE}_L + \mathrm{SSE}_R = 0.5 + 0.5 = 1.$$

Improvement

The split reduces training SSE by 49. Different leaves predict different local constants.

Range

A basic regression tree cannot extrapolate a trend beyond observed regions. Its leaf value remains a training-derived constant.

Regression thresholds compare the full child error

Threshold	Left mean	Right mean	Total SSE
1.5	1	19/3	86/3
2.5	1.5	8.5	1
3.5	11/3	9	86/3

Same four targets

Ordered inputs 1, 2, 3, 4 have targets 1, 2, 8, 9. The parent mean 5 gives SSE50.

Winning threshold

At 2.5, each child holds two close targets and has SSE0.5. The sum 1 is much smaller than 28.667 for either edge split.

Aggregation

Use the sum of within-child squared deviations when comparing this SSE definition. Refit each child's mean rather than keeping the parent's value 5.

A depth-two tree represents XOR

Path	Prediction
$x_1 \leq 0.5$, $x_2 \leq 0.5$	-1
$x_1 \leq 0.5$, $x_2 > 0.5$	$+1$
$x_1 > 0.5$, $x_2 \leq 0.5$	$+1$
$x_1 > 0.5$, $x_2 > 0.5$	-1

Representation

Two sequential questions create four pure leaves. One linear score in the original two features cannot do this.

Greedy learning

Either root split leaves both children 50% positive, so its immediate gain is zero. A learner that stops at zero gain will not discover this exact tree.

Connection

Expressive capacity and the success of a training procedure are separate questions.

A zero-gain root can unlock informative descendants

XOR root

Either feature split creates two children, each with one positive and one negative. Parent and weighted child Gini are both 0.5, so immediate gain is 0.

One more level

Testing the other feature within each child produces four pure leaves, reducing whole-tree Gini from 0.5 to 0.

Two configurations

A strict positive-gain stopping rule keeps a root leaf. Allowing a zero-gain root with depth 2 permits the representable XOR tree.

Tradeoff

Allowing weak initial splits can reveal interactions or create noise-driven branches. Validation and complexity constraints decide whether that freedom is useful.

A perfectly fitted tree can follow label noise

Growing without restriction

A tree can keep splitting until each leaf holds one example. Training impurity then reaches zero even when labels contain errors.

One mistaken label

A small extra branch isolates one mislabeled patient. It fixes one training error but creates a narrow rule that may fail on similar new patients.

Bias and variance

A shallow tree can miss real structure. A deep tree can react strongly to small data changes. Complexity must match the available evidence.

Objective

Training impurity alone rewards more partitions. Validation estimates whether those partitions improve prediction for new examples.

Tree depth and average prediction cost are different

Illustrative routing

70% of future-like validation cases stop after the root test. The remaining 30% take three tests before reaching a leaf.

$$\mathbb{E}[\text{tests}] = 0.7(1) + 0.3(3) = 1.6.$$

Maximum depth

The longest path requires three tests, while the average case requires 1.6. These summarize different costs.

Design implication

An early cheap measurement can avoid expensive later tests for many cases. Measure actual routing and acquisition cost on representative observations.

Evidence

The 70/30 proportions are an illustrative workload, not guaranteed by tree geometry.

Prepruning controls growth before a branch appears

Setting	Effect on a candidate tree
`max_depth`	Limits the number of sequential tests.
`min_samples_leaf`	Rejects splits that create a child with too little data.
`min_impurity_decrease`	Requires a sufficient decrease in the training criterion.

Ten samples

With minimum leaf size 4, a split into 2 and 8 samples is inadmissible even if the small child is pure.

Tradeoff

Early stopping can remove noise or block a useful later split. The XOR root illustrates the latter risk. Choose settings using validation predictions.

Postpruning trades impurity for fewer leaves

$$R_\alpha(T) = R(T) + \alpha|L(T)|, \qquad R(T) = \sum_{t \in L(T)} \frac{n_t}{n} I(D_t).$$

Terms

$R(T)$ is sample-weighted leaf impurity. $|L(T)|$ is the number of leaves. $\alpha \geq 0$ charges for each leaf.

Two candidate trees

Tree A: $R = 0.10$, 6 leaves. Tree B: $R = 0.16$, 3 leaves. Their costs are $0.10 + 6\alpha$ and $0.16 + 3\alpha$.

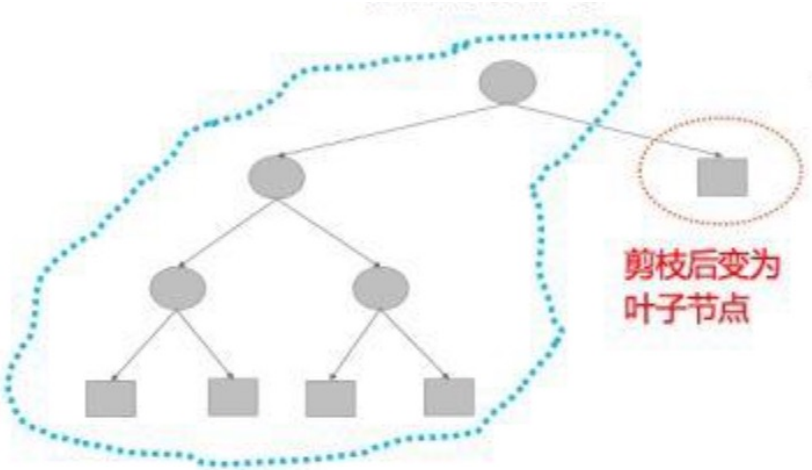
Decision

A wins when $\alpha < 0.02$. B wins when $\alpha > 0.02$. They tie at $\alpha = 0.02$.

Units

Use one consistent normalization of R. Count-weighted and fraction-weighted definitions produce differently scaled alpha values.

Postpruning trades impurity for fewer leaves



Three candidate trees exchange fit for complexity

Alpha	6-leaf:R=0.10	3-leaf:R=0.16	1-leaf:R=0.30
0.01	0.16	0.19	0.31
0.03	0.28	0.25	0.33
0.10	0.70	0.46	0.40

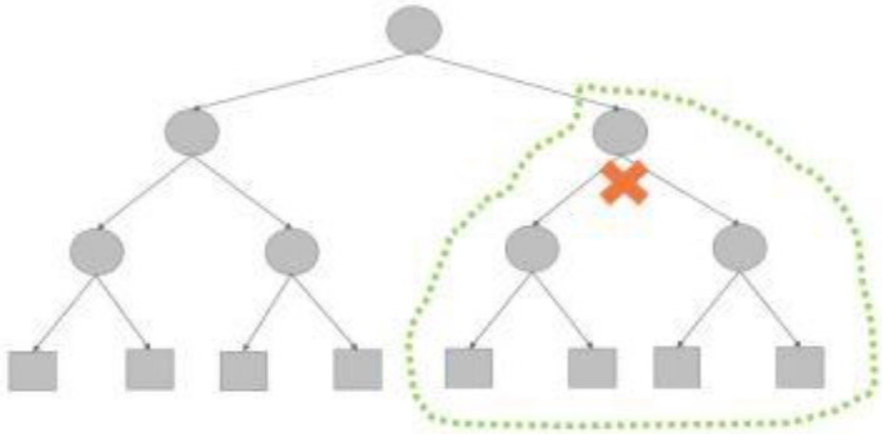
Winner

The six-leaf tree wins at 0.01, the three-leaf tree at 0.03 and the single leaf at 0.10. Each cell is $R + \alpha L$.

Breakpoints

Six versus three leaves tie at 0.02. Three versus one tie at 0.07. Compare all candidates because an intermediate tree can dominate a pairwise intersection.

Pruning removes a complete subtree



A branch has a calculable pruning threshold

Local comparison

Replace subtree T_t with a single leaf t. Costs outside that subtree cancel.

$$R(T_t) + \alpha |L(T_t)| = R(t) + \alpha \quad \Rightarrow \quad \alpha_{\text{eff}} = \frac{R(t) - R(T_t)}{|L(T_t)| - 1}.$$

Calculation

A 3-leaf branch contributes impurity 0.06. Replacing it with one leaf gives 0.14. The threshold is $(0.14 - 0.06)/(3 - 1) = 0.04$.

Reading

Above 0.04, saving two leaves outweighs the 0.08 impurity increase. After pruning, effective thresholds must be recomputed on the remaining tree.

The weakest-link comparison starts with local branches

Two branches

BranchA has 3 leaves, impurity 0.06 and collapsed impurity 0.14. BranchB has 2 leaves, impurity 0.02 and collapsed impurity 0.07.

$$\alpha_A = (0.14 - 0.06)/(3 - 1) = 0.04, \quad \alpha_B = (0.07 - 0.02)/(2 - 1) = 0.05.$$

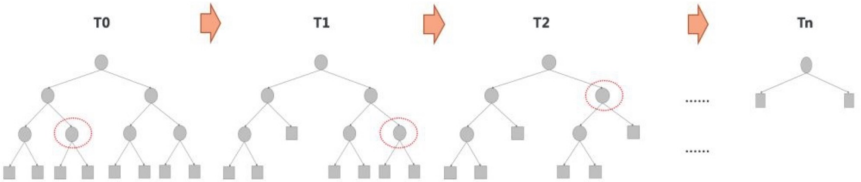
First candidate

A loses its extra-leaf advantage at the smaller alpha, so the path first collapsesA. Recompute affected ancestor thresholds after that change.

Units

All contributions use one whole-tree impurity normalization. Comparing raw local impurity with globally weighted impurity would mix different costs.

A pruning path supplies nested candidates



Training produces candidates

$T_0, T_1, T_2, \ldots$ have fewer leaves. Each tree keeps part of the earlier structure. The training data supply effective alpha values.

Validation chooses the subtree

Compare candidate predictions on separate validation data and choose alpha. The pruning path's training cost is not an unbiased estimate of future performance.

Selected model

Keep the chosen rules and leaf counts. A simpler tree is useful when it retains enough predictive accuracy.

Validation chooses complexity on unseen examples

Candidate	Leaves	Training accuracy	Validation accuracy
Deep	24	1.00	0.84
Medium	10	0.97	0.89
Shallow	3	0.88	0.85

Interpretation

The medium candidate wins on these validation observations. Its lower training accuracy is the price of removing unstable details.

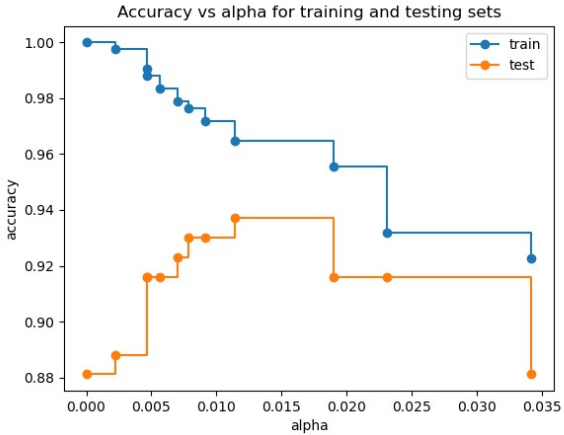
Fair selection

Compare identical splits and task metrics. Repeat folds when data are limited. Refit the chosen setting on training data and evaluate the reserved test set.

Numbers

This table is a teaching example, not a measured result from the course code. Small score differences need variability estimates.

Pruning trades training fit against held-out performance



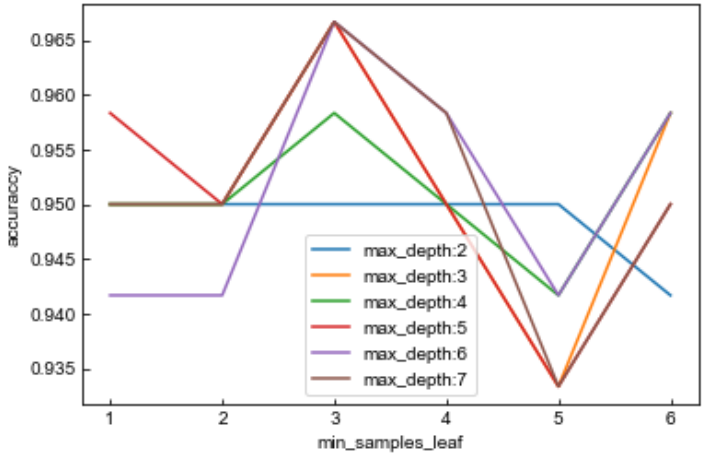
Tree training exposes interpretable parameters

```
tree = DecisionTreeClassifier(
    max_depth=3, min_samples_leaf=5,
    criterion="gini", random_state=0)
tree.fit(X_train, y_train)
pred = tree.predict(X_valid)
print(export_text(tree, feature_names=names))
path = tree.cost_complexity_pruning_path(
    X_train, y_train)
alphas = path.ccp_alphas
```

Meaning

Depth 3 and leaf size 5 restrict growth. The pruning path uses training examples. Candidate alphas still need validation comparison. Exported rules refer to the actual input columns.

Tree training exposes interpretable parameters



Tiny leaves produce fragile probabilities

Empirical probability

A leaf with 2 positive and 0 negative examples reports $\hat{p} = 1$. A leaf with 90 positive and 10 negative examples reports $\hat{p} = 0.9$.

Laplace smoothing

With C classes, use $\tilde{p}_c = (n_c + \lambda)/(n + C\lambda)$. For $(2, 0)$ and $\lambda = 1$, the positive probability becomes $3/4$.

What changes

Smoothing reduces extreme estimates from small counts. Larger leaf size supplies more observations. Calibration compares predicted probabilities with held-out outcome frequencies.

Limits

Smoothing does not create missing evidence or guarantee calibration. Evaluate the full fitted model on new observations.

Smoothing changes small-count certainty most strongly

Counts(+, -)	Empiricalp	Add-onep
1, 0	1.000	2/3
9, 1	0.900	10/12
90, 10	0.900	91/102

One consistent rule

For two classes, add 1 to each count:$\tilde{p}_+ = (n_+ + 1)/(n + 2)$.

Sample-size effect

A single positive moves from 1 to 0.667. Ninety positives among 100 move from 0.9 to about 0.892. The same prior convention has a smaller effect when evidence is larger.

Use

Smoothing tempers estimates. Calibration still compares predicted probabilities with actual held-out frequencies.

Class weights change the learning priorities

Rare positive class

With 95 negatives and 5 positives, predicting negative for every example gives 95% accuracy and zero positive recall.

Weighted counts

A positive weight of 10 turns leaf counts (n_+, n_-) into weighted counts $(10n_+, n_-)$. The split criterion uses the resulting proportions.

One leaf

Raw counts $(2, 8)$ give positive proportion 0.2. Weighted counts $(20, 8)$ give $20/28 \approx 0.714$.

Probability

Weighted proportions reflect the training objective. They need not equal the unweighted population probability. Compare recall, precision and relevant costs on held-out data.

A confusion table makes minority-class behavior visible

Actual/predicted	Positive	Negative
Positive	4	1
Negative	8	87

$$\text{accuracy} = 91/100 = 0.91, \quad \text{recall} = 4/5 = 0.8, \quad \text{precision} = 4/12 = 1/3.$$

Interpretation

The model finds four of five positives, but eight negative cases also trigger positive predictions. Accuracy alone hides that workload.

Comparison

An all-negative baseline reaches 95% accuracy yet has zero positive recall. Select the task metric before comparing models.

Class weighting does not create new observations

One leaf

Raw counts 4 positive and 16 negative give empirical positive fraction 0.2.

Positive weight 4

Weighted counts become 16 and 16. The weighted positive fraction is 0.5.

Positive weight 10

Weighted counts 40 and 16 give fraction $40/56 \approx 0.714$. The actual leaf still contains 20 observations, including only four positives.

Decision meaning

Weights change training priorities and split selection. They do not multiply the amount of real evidence or guarantee a population probability. Evaluate resulting recall and false positives on unweighted held-out cases.

Feature importance describes one fitted model

Impurity importance

Add the weighted impurity decreases for splits using a feature, then normalize across features. It summarizes how this fitted tree used the training data.

Bias

A feature with many possible cut points can win some splits by chance. Correlated substitutes can share or exchange importance. Importance does not establish causation.

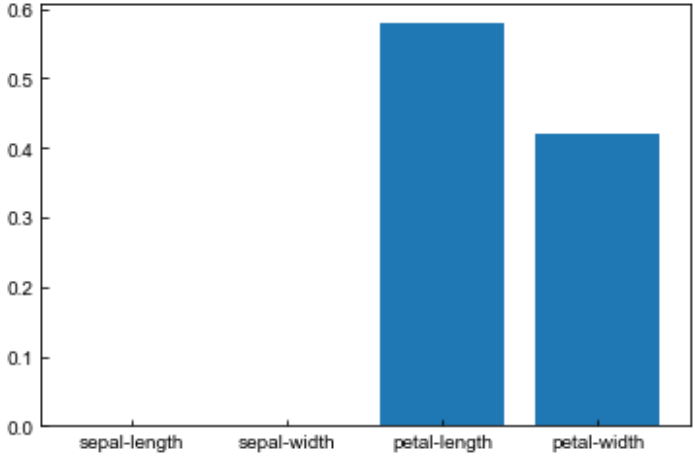
Held-out check

Permutation importance measures how much a chosen validation metric worsens after shuffling one column. Shuffling a correlated feature may produce unrealistic combinations.

Question

Would removing this measurement preserve predictive performance? Test a retrained model with and without it.

Feature importance describes one fitted model



Small data changes can change a tree

Mechanism

If two root candidates have similar gains, changing one observation can reverse their order. All later descendants then see different subsets.

Near tie

Candidate gains 0.081 and 0.079 differ by 0.002. A small change in counts can select another root even if overall accuracy remains similar.

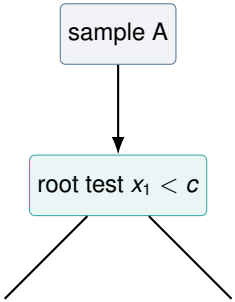
Checks

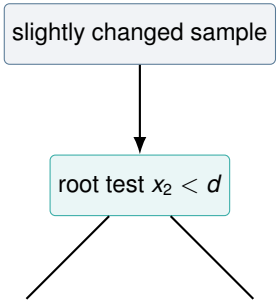
Refit across resampled training sets. Compare predictions as well as selected features and rule thresholds. A stable metric does not imply stable explanations.

Course connection

Bagging averages predictions from varied trees. Random forests also vary candidate feature sets. Their detailed treatment follows in the ensemble lectures.

Small data changes can change a tree





A small data change can alter an early split and many downstream predictions.

Prediction agreement and rule agreement measure different stability

Refit	Root feature	Threshold	Validationaccuracy
A	Petallength	2.45	0.89
B	Petalwidth	0.80	0.90
C	Petallength	2.50	0.89

Illustrative comparison

Different roots can give similar metrics. Measure how often predictions agree on the same held-out rows, as well as how much the rules change.

Explanation use

A rule intended for human decisions needs repeatable conditions. A model used only for prediction can tolerate alternative rules if performance and behavior remain reliable.

A fair comparison uses the same observations

Question	Perceptron	Tree
Input geometry	Scale affects updates	Threshold order matters
Nonlinear patterns	Need useful transformed features	Combine conditional regions
Training stop	Zero-update pass or budget	Growth restrictions and pruning
Main check	Representation and separability	Complexity and stability

Comparison design

Reserve identical test observations. Fit preprocessing only on training data. Choose settings through validation. Compare suitable metrics and inspect the same mistakes.

Result interpretation

A model wins for a particular dataset and representation. The comparison does not rank the two methods for every task.

A small measurement change can cross a learned boundary

Two models

The perceptron $a = -2x_1 + 1$ and a tree asking $x_1 \leq 0.5$ agree away from the threshold on the four-point data.

Nearby measurements

At $x_1 = 0.49$, the score is 0.02 and the tree takes its left branch. At 0.51, the score is -0.02 and the tree takes its right branch.

Question

If the instrument uncertainty is ± 0.02, the observed class can flip near the threshold. Agreement between models does not remove measurement uncertainty.

Response

Check measurement precision and held-out behavior near the boundary. A score near zero is a location statement, not automatically a calibrated risk probability.

The same split supports both model pipelines

```
linear = make_pipeline(
    StandardScaler(),
    Perceptron(max_iter=1000, random_state=0))
tree = DecisionTreeClassifier(
    max_depth=3, min_samples_leaf=5, random_state=0)
for model in [linear, tree]:
    model.fit(X_train, y_train)
    print(balanced_accuracy_score(
        y_valid, model.predict(X_valid)))
```

Interpretation

Both models see identical rows. Only the linear pipeline learns scaling here. Tune settings using training and validation data, then compare final test predictions.

A training loop can fail despite a correct-looking formula

BugA

Labels remain 0 and 1 in the handwritten update $w \leftarrow w + \eta yx$. Negative-class rows with $y = 0$ never move the weights.

BugB

The code updates w but not b, so it runs a different hypothesis class and cannot shift the boundary freely.

BugC

The code breaks after the first correctly classified row rather than a full zero-update epoch.

Repair check

Use the known four-point dataset, log scores and updates, and verify the final $(-2, 0, 1)$ for the specified order and tie rule. Distinguish a failed implementation from an inseparable dataset.

Exercise: a larger minimum leaf size changes the winner

Use the six-fruit data

Set minimum leaf size to 3. Examine width thresholds 6.25, 6.55, 6.75, 6.95 and height thresholds 6.30, 8.05, 9.55.

Tasks

Identify which candidates are admissible. For each remaining candidate, calculate weighted Gini and compare with the unrestricted winner 6.55.

Reminder

The unrestricted width split produced two and four observations. A growth constraint can reject it before comparing impurity.

Explain the cost

Say how the new restriction trades detailed fit for more evidence in every leaf. It does not change the raw measurements or labels.

Answer: admissibility precedes impurity ranking

Candidate	Children sizes	Allowed with leaf size 3
Width 6.25 /6.55 /6.95	1/5;2/4;5/1	No
Width 6.75	3/3	Yes
Height 6.30 /8.05 /9.55	1/5;4/2;5/1	No

Remaining split

Width 6.75 creates $(1O, 2L)$ and $(2O, 1L)$. Each has Gini $4/9$, so weighted Gini remains $4/9 \approx 0.4444$.

Tradeoff

Gain is only $0.5 - 4/9 = 1/18$. This is weaker than unrestricted gain 0.25, but no leaf relies on fewer than three cases.

Exercise: decide whether a four-leaf branch should remain

Subtree evidence

A four-leaf branch contributes training impurity 0.03. Collapsing it into one leaf contributes 0.15. All other parts of the tree stay unchanged.

Tasks

Calculate its effective alpha. Compare retained and collapsed costs at $\alpha = 0.03$ and 0.05. Explain which branch is cheaper each time.

Further question

Even after the training cost comparison, why is a separate validation comparison needed to choose the final alpha?

Units

Use the same globally weighted impurity definition for 0.03 and 0.15. Count the collapsed leaf rather than deleting the entire region.

Answer: alpha 0.04 is this branch's break-even point

$$\alpha_{\text{eff}} = (0.15 - 0.03)/(4 - 1) = 0.04.$$

Alpha	Retain:0.03+4 alpha	Collapse:0.15+alpha	Cheaper
0.03	0.15	0.18	Retain
0.05	0.23	0.20	Collapse

Meaning

Saving three leaves costs 0.12 in training fit. At alpha 0.04 those costs balance. Larger penalties favor collapsing this branch.

Final decision

The pruning path supplies candidates. Held-out predictions choose the useful complexity for the task, including minority-class and probability behavior.

Exercise combines an update and a split

Part A: perceptron

$w = (1, -1), b = -0.5, x = (0, 2), y = +1, \eta = 0.5$. Calculate the signed score and updated w, b. Then check the new score of the same point.

Part B: tree

A parent has counts $(4, 4)$. Candidate 1 creates children $(3, 1), (1, 3)$. Candidate 2 creates $(2, 0), (2, 4)$. Calculate each weighted child Gini and choose the larger gain.

Part C: explain

Why can a tree represent XOR while a greedy learner still stop at its root? Why can a pure two-sample leaf be unreliable?

Working method

For A, retain the bias. For B, retain the child-size weights. For C, distinguish representation from evidence.

Exercise answers connect formulas to behavior

A

$a = -2.5$. The update gives $w' = (1, 0), b' = 0$. Its new score is 0, an improvement of 2.5, but it remains a training boundary case.

B

Parent Gini 0.5. Candidate 1: $G_{\text{after}} = 0.375$, gain 0.125. Candidate 2: $G_{\text{after}} = \frac{2}{8}(0) + \frac{6}{8}\frac{4}{9} = \frac{1}{3}$, gain $\frac{1}{6}$. Candidate 2 wins.

C

XOR needs a zero-gain first split before useful descendants appear. Two identical labels give a pure leaf but only two observations of evidence.

Next lecture

Changing features changes separability and available split questions. Feature engineering makes that input representation explicit.